

19 — 100 questions théoriques + 100 manip de code

Banque de révision massive pour la soutenance. Réponses courtes volontairement. Colonne de gauche = ce que le jury peut demander ; à droite = la réponse / la solution.

PARTIE 1 — 100 QUESTIONS THÉORIQUES

Architecture & général (1-12)

1. **C'est quoi l'architecture globale du projet ?** → Front statique (HTML/CSS/JS) servi par Nginx + une API Go + une base MySQL, le tout dans Docker.
2. **Pourquoi séparer frontend et backend ?** → Chacun évolue indépendamment ; le back expose une API, le front la consomme.
3. **Nginx sert à quoi chez vous ?** → Servir les fichiers statiques ET faire proxy `/api/` vers le backend Go.
4. **Pourquoi Docker ?** → Même environnement partout (dev, VM), déploiement reproductible.
5. **C'est quoi une API REST ?** → Une interface HTTP où chaque URL = une ressource, et les verbes (GET/POST/PUT/DELETE) = les actions.
6. **Le front et le back sont sur le même serveur ?** → Oui en prod (même origine), Nginx route `/api/` vers Go.
7. **Combien de rôles utilisateurs ?** → 4 : Utilisateur, Pro, Salarié, Administrateur.
8. **Où est la logique métier ?** → Dans le backend Go (les handlers + les requêtes bdd).
9. **Le front peut-il accéder directement à MySQL ?** → Non, jamais. Il passe toujours par l'API.
10. **Qu'est-ce qui tourne sur quel port ?** → Backend Go : 8081 ; front (Nginx) : 80 ; MySQL : 3306.
11. **C'est quoi le langage du backend ?** → Go (Golang).
12. **Pourquoi Go plutôt que PHP/Node ?** → Compilé, rapide, typé, un seul binaire à déployer.

Go / backend (13-30)

1. **C'est quoi un handler en Go ?** → Une fonction `func(w http.ResponseWriter, r *http.Request)` qui traite une requête.

2. **Comment on déclare une route ?** → `http.HandleFunc("GET /chemin", handler)` .
3. **C'est quoi un middleware ?** → Une fonction qui enveloppe un handler pour faire un contrôle avant (ex. vérifier le token).
4. **Comment Go renvoie du JSON ?** → `json.NewEncoder(w).Encode(donnees)` .
5. **Comment lire le corps JSON d'une requête ?** → `json.NewDecoder(r.Body).Decode(&struct)` .
6. **C'est quoi une struct ?** → Un type qui regroupe des champs (comme un objet).
7. **À quoi servent les tags `json:"..."` ?** → Dire quel nom de clé JSON correspond à quel champ Go.
8. **Comment récupérer un paramètre d'URL ?** → `r.URL.Query().Get("id")` (query) ou `r.PathValue("id")` (chemin).
9. **Comment gérer les erreurs en Go ?** → On renvoie une `error` et on la teste : `if err != nil { ... }` .
10. **C'est quoi `defer` ?** → Reporte l'exécution d'une ligne à la fin de la fonction (ex. `defer rows.Close()`).
11. **Comment on fait une requête SQL ?** → `Db.Query(...)` (lecture), `Db.Exec(...)` (écriture), `Db.QueryRow(...)` (une ligne).
12. **C'est quoi le `?` dans les requêtes ?** → Un paramètre préparé (évite les injections SQL).
13. **Pourquoi utiliser des requêtes préparées ?** → Sécurité : empêche l'injection SQL.
14. **C'est quoi `rows.Scan()` ?** → Copie les colonnes d'une ligne SQL dans des variables Go.
15. **C'est quoi `COALESCE` dans vos `SELECT` ?** → Renvoie une valeur par défaut si la colonne est NULL.
16. **Comment compiler le backend ?** → `cd API && go build ./...` .
17. **C'est quoi un paramètre variadique ?** → `...string` : accepte 0, 1 ou plusieurs valeurs (ex. `rolesAutorises`).
18. **Comment on lit une variable d'environnement ?** → `os.Getenv("NOM")` .

JavaScript / frontend (31-42)

1. **Comment on appelle l'API depuis le front ?** → `fetch(url, options)` .
2. **C'est quoi `async/await` ?** → Une écriture lisible du code asynchrone (attendre une réponse sans bloquer).
3. **C'est quoi une Promise ?** → Un objet représentant un résultat futur (réussi ou échoué).
4. **Comment on stocke le token côté front ?** → Dans `localStorage` .
5. **Différence `localStorage` / `sessionStorage` ?** → `localStorage` persiste après fermeture ; `sessionStorage` non.
6. **Comment modifier le DOM ?** → `document.getElementById(...)` , `.innerHTML` , `.textContent` , `createElement` .
7. **C'est quoi `data-i18n` ?** → Un attribut pour la traduction : le texte est remplacé selon la langue.

8. **Comment on gère plusieurs langues ?** → `translate.js` + un dictionnaire ;
`appliquerTraductions()` remplace les textes.
9. **C'est quoi `addEventListener` ?** → Attache une fonction à un événement (clic, chargement...).
10. `DOMContentLoaded` **sert à quoi ?** → Lancer du code quand le HTML est prêt.
11. **Pourquoi `?v=2` sur un script ?** → Forcer le navigateur à recharger la nouvelle version (cache-busting).
12. **C'est quoi le CORS ?** → Une sécurité navigateur ; le backend autorise l'origine via des en-têtes `Access-Control-*`.

Base de données / SQL (43-54)

1. **Quel SGBD ?** → MySQL 8.
2. **C'est quoi une clé primaire ?** → Identifiant unique d'une ligne (`id`).
3. **C'est quoi une clé étrangère ?** → Une colonne qui référence la clé primaire d'une autre table.
4. **C'est quoi `ON DELETE CASCADE` ?** → Supprime automatiquement les lignes enfants quand le parent est supprimé.
5. **Différence INNER JOIN / LEFT JOIN ?** → INNER = seulement les correspondances ; LEFT = tout à gauche même sans correspondance.
6. **C'est quoi un index ?** → Une structure qui accélère les recherches sur une colonne.
7. **C'est quoi une transaction ?** → Un groupe d'opérations tout-ou-rien (COMMIT / ROLLBACK).
8. **Comment compter des lignes ?** → `SELECT COUNT(*) FROM table`.
9. **C'est quoi `GROUP BY` ?** → Regrouper des lignes pour agréger (COUNT, SUM...).
10. **C'est quoi un ENUM ?** → Une colonne à valeurs limitées (ex. `role`, `statut_vente`).
11. **Comment éviter les injections SQL ?** → Requêtes préparées avec `?`, jamais de concaténation.
12. **Comment sauvegarder la base ?** → `mysqldump` ; restaurer avec `mysql < fichier.sql`.

Sécurité / JWT / bcrypt (55-70)

1. **Comment stockez-vous les mots de passe ?** → Hachés avec bcrypt (coût 10), jamais en clair.
2. **Peut-on déchiffrer un hash bcrypt ?** → Non ; on compare seulement (`CompareHashAndPassword`).
3. **C'est quoi un JWT ?** → Un jeton signé contenant des infos (id, rôle, expiration).
4. **Le JWT est-il chiffré ?** → Non, il est **signé** (lisible mais infalsifiable sans la clé).
5. **Comment le serveur vérifie un token ?** → Il revérifie la signature avec sa clé secrète (`jwtKey`).
6. **Différence authentification / autorisation ?** → Auth = qui es-tu (token) ; autorisation = as-tu le droit (rôle).
7. **Différence 401 / 403 ?** → 401 = pas/mauvais token ; 403 = connecté mais pas les droits.
8. **Où est stocké le rôle ?** → Dans le token (signé), pas envoyé à la main par le front.

9. **Peut-on tricher en changeant son rôle ?** → Non : la signature ne correspond plus → token invalide → 401.
10. **Combien de temps vit un token ?** → 24 h (`ExpiresAt`).
11. **Que se passe-t-il à l'expiration ?** → `token.Valid` = faux → 401 → redirection login.
12. **C'est quoi le middleware `VerifyTokenMiddleware` ?** → Vérifie que l'utilisateur est connecté (token valide).
13. **C'est quoi `VerifyRoleMiddleware` ?** → Vérifie en plus que le rôle est autorisé.
14. **La sécurité front (`auth_guard`) suffit-elle ?** → Non, c'est du confort ; la vraie sécurité est backend.
15. **C'est quoi HS256 ?** → Algorithme de signature symétrique du JWT (une seule clé secrète).
16. **Comment protéger la clé JWT ?** → Variable d'environnement, jamais commitée.

Stripe / paiement (71-80)

1. **Comment gérez-vous les paiements ?** → Via Stripe ; on ne manipule aucune donnée bancaire.
2. **C'est quoi Stripe Connect ?** → Chaque vendeur a un compte Stripe relié ; il reçoit l'argent directement.
3. **Comment prenez-vous une commission ?** → `ApplicationFeeAmount` (5 %) + `TransferData` vers le vendeur.
4. **C'est quoi une Checkout Session ?** → La page de paiement hébergée par Stripe.
5. **Pourquoi les montants sont ×100 ?** → Stripe travaille en centimes (25 € = 2500).
6. **C'est quoi un webhook ?** → Stripe appelle notre serveur pour confirmer un paiement.
7. **Pourquoi un webhook et pas la page de succès ?** → Le client peut fermer son navigateur ; le webhook arrive toujours.
8. **Comment vérifie-t-on que le webhook vient de Stripe ?** → Vérification de la signature avec `WebhookSecret` .
9. **Êtes-vous en vrai argent ?** → Non, mode test (`sk_test_`), carte `4242 4242 4242 4242` .
10. **Quels types de paiement gérez-vous ?** → Achat d'annonce, inscription événement, abonnement Pro récurrent.

Docker / déploiement (81-90)

1. **C'est quoi une image Docker ?** → Un modèle figé de l'application (code + dépendances).
2. **C'est quoi un conteneur ?** → Une instance qui tourne à partir d'une image.
3. **C'est quoi docker-compose ?** → Un fichier qui décrit/lance plusieurs conteneurs ensemble.
4. **C'est quoi un Dockerfile ?** → La recette pour construire une image.
5. **Comment reconstruire un service ?** → `docker compose up -d --build <service>` .

6. **Comment voir les logs ?** → `docker compose logs -f <service>`.
7. **C'est quoi un volume ?** → Un stockage persistant (ex. les uploads, la base) hors du conteneur.
8. **Pourquoi un volume pour MySQL ?** → Garder les données même si le conteneur est recréé.
9. **Comment passer une config au conteneur ?** → Variables d'environnement (`environment:` dans `compose`).
10. **Comment déployez-vous une nouvelle version ?** → Build image → push → `pull` + `up -d` sur la VM.

HTTP / API / divers (91-100)

1. **Différence GET / POST ?** → GET lit (paramètres dans l'URL) ; POST envoie des données (dans le corps).
 2. **À quoi sert PUT ?** → Modifier une ressource existante.
 3. **À quoi sert DELETE ?** → Supprimer une ressource.
 4. **C'est quoi un code 200 / 404 / 500 ?** → 200 OK, 404 introuvable, 500 erreur serveur.
 5. **C'est quoi une requête preflight OPTIONS ?** → Le navigateur demande l'autorisation avant un PUT/DELETE (CORS).
 6. **Comment les notifications marchent ?** → Une ligne en base (cloche) + un push OneSignal, via `SendPushNotification`.
 7. **Comment le forum s'actualise en temps réel ?** → Polling : `setInterval` recharge les messages toutes les 4 s.
 8. **C'est quoi le Swagger ?** → Une doc interactive de l'API (`/swagger` , basée sur `openapi.yaml`).
 9. **Comment testez-vous une route protégée ?** → `curl` avec l'en-tête `Authorization: Bearer <token>`.
 10. **Quelle est la partie la plus complexe ?** → Le paiement Stripe (Connect + commission + webhook).
-

PARTIE 2 — 100 MANIPS DE CODE

Format : **objectif** → **fichier(s)** → **solution courte**. Fais-les sur ton environnement local.

Frontend — HTML / CSS (1-20)

1. Changer la couleur d'accent des annonces → `style/annonceAll.css` → `:root{--ac:255,120,0;}`.

2. Ajouter un lien nav « À propos » → une page HTML → `<a class="nav-link" href="/a-propos">À propos</a>` .
3. Changer le titre d'un onglet → `<title>` de la page → nouveau texte.
4. Mettre le logo plus grand → CSS du `.nav-logo img` → `height:48px;` .
5. Changer le texte d'un bouton → le HTML du bouton → nouveau libellé.
6. Ajouter un footer → fin du `<body>` → `<footer>...</footer>` .
7. Arrondir les cartes annonces → `.listing-card` → `border-radius:16px;` .
8. Changer la police → `<link>` Google Fonts + `font-family` dans le CSS.
9. Masquer un élément → CSS → `display:none;` .
10. Ajouter une ombre aux cartes → `.card` → `box-shadow:0 4px 12px rgba(0,0,0,.3);` .
11. Rendre la nav collante → `.topnav` → `position:sticky; top:0;` .
12. Changer le nombre de colonnes de la grille → `.listings-grid` → `grid-template-columns:repeat(4,1fr);` .
13. Ajouter un `placeholder` à un champ → l'input → `placeholder="Votre texte"` .
14. Rendre un champ obligatoire (HTML) → l'input → `required` .
15. Changer la couleur du thème Pro → `style/client.css` `.theme-pro` → modifier `--blue` .
16. Ajouter une icône Font Awesome → `<i class="fas fa-star"></i>` .
17. Centrer un bloc → CSS → `margin:0 auto; max-width:...;` .
18. Changer le favicon → `<link rel="icon" href="...">` .
19. Ajouter un espace entre sections → CSS → `padding / margin` .
20. Passer un texte en gras → CSS → `font-weight:700;` .

Frontend — JavaScript (21-45)

1. Changer le nombre d'annonces par page → `annonceAll.js` → `ANN_PAR_PAGE = 12` .
2. Changer le nombre d'utilisateurs par page → `admin/user.js` → `USERS_PAR_PAGE = 20` .
3. Logguer le nombre de pages → `renderPageAnnonces` → `console.log(nbPages)` .
4. Afficher une alerte au chargement → `document.addEventListener("DOMContentLoaded", ()=>alert("ok"))` .
5. Trier les annonces par prix croissant → appeler `sortListings('price-asc')` .
6. Filtrer les annonces gratuites → filtrer `allAnnonces` SUR `prix<=0` .
7. Ajouter un bouton « retour en haut » → `window.scrollTo({top:0})` .
8. Afficher le nombre de résultats → `resultCount.textContent = items.length` .
9. Changer l'intervalle de polling forum → `forum.js` → `setInterval(...,2000)` .
10. Rediriger si non connecté → `if(!localStorage.getItem("token")) location.href="/login"` .
11. Récupérer l'id utilisateur → `localStorage.getItem("userId")` .

12. Ajouter un header Authorization à un fetch → `headers:{Authorization:"Bearer "+token}` .
13. Afficher une erreur si le fetch échoue → `.catch(err=>alert("Erreur"))` .
14. Formater un prix avec « € » → ``${prix} €`` .
15. Vider un conteneur → `element.innerHTML = ""` .
16. Créer une carte dynamiquement → `document.createElement("div")` + `appendChild` .
17. Ajouter un écouteur sur un bouton → `btn.addEventListener("click",fn)` .
18. Changer la langue par défaut → `translate.js` (langue initiale).
19. Désactiver un bouton → `btn.disabled = true` .
20. Faire défiler vers un élément → `el.scrollToView({behavior:"smooth"})` .
21. Empêcher le rechargement d'un form → `e.preventDefault()` .
22. Lire la valeur d'un input → `document.getElementById("x").value` .
23. Ajouter une classe conditionnellement → `el.classList.toggle("on", condition)` .
24. Recharger la page après une action → `window.location.reload()` .
25. Construire un FormData → `const fd=new FormData(); fd.append("titre",...)` .

Backend Go (46-70)

1. Créer un endpoint `GET /admin/users/count` → route + handler + requête `COUNT(*)` .
2. Passer la commission de 5 % à 10 % → `admin/stripe.go` → `*10)/100` et `0.10` .
3. Réserver une route aux Admins → `route/...` → `VerifyRoleMiddleware(handler,"Administrateur")` .
4. Autoriser Admin OU Salarié → `VerifyRoleMiddleware(handler,"Administrateur","Salarié")` .
5. Ajouter un champ `telephone` renvoyé → `models/users.go` + `SELECT` + `Scan` .
6. Refuser une annonce sans titre → `if r.FormValue("titre")=="">{http.Error(...,400);return}` .
7. Changer la durée du token (12 h) → `jwt.go` → `Add(12 * time.Hour)` .
8. Ajouter un log dans un handler → `fmt.Println("appel handler X")` .
9. Renvoyer un JSON custom → `json.NewEncoder(w).Encode(map[string]string{"ok":"1"})` .
10. Lire un paramètre d'URL → `r.URL.Query().Get("id")` .
11. Lire un param de chemin → `r.PathValue("id")` .
12. Ajouter un endpoint `GET /admin/stats` → route + handler renvoyant `{users,annonces}` .
13. Créer une nouvelle table via une requête → `Db.Exec("CREATE TABLE ...")` .
14. Ajouter une validation de rôle inconnue → renvoyer 400 si rôle non prévu.
15. Trier une requête SQL → ajouter `ORDER BY date_creation DESC` .
16. Limiter une requête → ajouter `LIMIT 10` .
17. Filtrer par statut → `WHERE statut_vente = ?` .
18. Ajouter un `COALESCE` sur un champ nullable → `COALESCE(ville,'')` .
19. Gérer la méthode OPTIONS → `if r.Method=="OPTIONS"{w.WriteHeader(200);return}` .

20. Renvoyer 404 si introuvable → `http.Error(w, "introuvable", http.StatusNotFound)` .
21. Hacher un mot de passe → `auth.HashPassword(mdp)` .
22. Comparer un mot de passe → `bcrypt.CompareHashAndPassword(hash, saisi)` .
23. Ajouter un champ à une struct → nouveau champ + tag `json` .
24. Lire une variable d'env avec défaut → helper `envOr("NOM", "default")` .
25. Ajouter une route DELETE → `http.HandleFunc("DELETE /admin/x/{id}", handler)` .

SQL / Base de données (71-85)

1. Ajouter une colonne → `ALTER TABLE utilisateur ADD COLUMN telephone VARCHAR(20);` .
2. Supprimer une colonne → `ALTER TABLE x DROP COLUMN y;` .
3. Compter les annonces par catégorie → `GROUP BY c.id` avec `COUNT(a.id)` .
4. Valider un utilisateur → `UPDATE utilisateur SET validation='Validé' WHERE id=42;` .
5. Compter les annonces sponsorisées → `SELECT COUNT(*) FROM annonce WHERE is_sponsored=1;` .
6. Trouver les 5 dernières annonces → `ORDER BY id DESC LIMIT 5;` .
7. Supprimer les annonces d'un user → `DELETE FROM annonce WHERE id_user=?;` .
8. Masquer un message forum → `UPDATE message_forum SET est_modere=1 WHERE id_message=?;` .
9. Lister les users par rôle → `SELECT * FROM utilisateur WHERE role='Pro';` .
10. Créer un index → `CREATE INDEX idx_ville ON annonce(ville);` .
11. Compter les users → `SELECT COUNT(*) FROM utilisateur;` .
12. Mettre à jour un prix → `UPDATE annonce SET prix=10 WHERE id=?;` .
13. Sauvegarder la base → `mysqldump -u.. -p.. pa2026 > b.sql` .
14. Restaurer la base → `mysql -u.. -p.. pa2026 < b.sql` .
15. Voir la structure d'une table → `DESCRIBE annonce;` .

Docker / debug / outils (86-100)

1. Reconstruire le backend → `docker compose up -d --build backend` .
2. Voir les logs backend → `docker compose logs -f backend` .
3. Entrer dans le conteneur MySQL → `docker exec -it uc_mysql mysql -u.. -p..` .
4. Lister les conteneurs → `docker ps` .
5. Redémarrer un service → `docker compose restart backend` .
6. Voir les variables d'env d'un conteneur → `docker exec uc_backend env` .
7. Vérifier qu'un port écoute → `netstat -ano | grep 8081` .
8. Arrêter tous les conteneurs → `docker compose down` .
9. Reconstruire le front → `docker build -t upcycle-frontend ./Frontend` .

10. Compiler le back sans lancer → `cd API && go build ./... .`
 11. Tester une route au curl → `curl http://localhost:8081/... -H "Authorization: Bearer $TOKEN" .`
 12. Provoquer un 500 et lire l'erreur → mauvais param + `docker compose logs backend .`
 13. Vider le cache navigateur → Ctrl+Shift+R.
 14. Ouvrir la console front → F12 → onglet Console.
 15. Voir les requêtes réseau → F12 → onglet Réseau (Network).
-

Comment s'entraîner

- **J-3** : lis les 100 questions, coche celles que tu ne sais pas.
- **J-2** : refais les manips 21-24, 46-52, 71-74 (les plus demandées).
- **J-1** : relis seulement tes cases non cochées + les fiches 15/16/18.
- **Jour J** : garde ce document ouvert, il sert d'aide-mémoire.